

UNIVERZITET U BEOGRADU – ELEKTROTEHNIČKI
FAKULTET
KATEDRA ZA ELEKTRONIKU

DIGITALNA ELEKTRONIKA 1

Materijali za računske vežbe

**SINTEZA KOMBINACIONIH MREŽA POMOĆU KOLA
NISKOG STEPENA INTEGRACIJE I STATIČKI HAZARDI**

Vežbe 1

Autori:

Nikola Petrović p.z.nikola@etf.bg.ac.rs
Haris Turkmanović haris@etf.bg.ac.rs

Beograd, 20. novembar 2025.

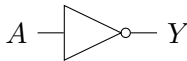
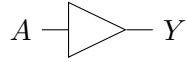
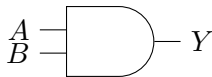
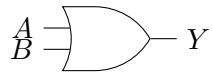
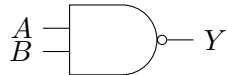
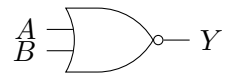
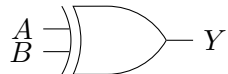
Sadržaj

1	Uvod	2
2	Zadaci	4
2.1	Zadatak 1	4
2.2	Zadatak 2	9
2.3	Zadatak 3	12
2.4	Zadatak 4	15
2.5	Zadatak 5	18
2.6	Zadatak 6	21
3	Zadaci za samostalni rad	24
3.1	Zadatak 1	24
3.2	Zadatak 2	24

1 Uvod

U okviru prvog dela ovih računskih vežbi obrađivaćemo praktične primere vezane za sintezu kombinacionih mreža korišćenjem kola niskog stepena integracije. Pod kolima niskog stepena integracija, za potrebe ovog kursa, podrazumevaćemo grupu osnovnih logičkih kola predstavljenih u tabeli 1.

Tabela 1: Pregled osnovnih logičkih kola

Naziv	Simbol	Funkcija	Tabela istinitosti															
Invertor (NOT)		$Y = \neg A = \bar{A}$	<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	Y	0	1	1	0									
A	Y																	
0	1																	
1	0																	
Bafer (BUFFER)		$Y = A$	<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	Y	0	0	1	1									
A	Y																	
0	0																	
1	1																	
I (konjunkcija, AND)		$Y = A \wedge B = A \cdot B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	0	1	0	0	1	1	1
A	B	Y																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
ILI (disjunkcija, OR)		$Y = A \vee B = A + B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	1
A	B	Y																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NI (NAND)		$Y = \neg(A \wedge B) = \overline{A \cdot B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	1	1	0	1	1	1	0
A	B	Y																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NILI (NOR)		$Y = \neg(A \vee B) = \overline{A + B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	0
A	B	Y																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Ekskluzivno ILI (XOR)		$Y = A \oplus B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	0
A	B	Y																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Ekvivalencija (XNOR)		$Y = \neg(A \oplus B) = \overline{A \oplus B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	1
A	B	Y																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Jedan od bitnijih koraka u procesu sinteze kombinacionih mreža predstavlja proces minimizacije logičkih funkcija. Iako su razvijene različite metode koje imaju za cilj da minimizuju broj logičkih kola potrebnih za sintezu neke logičke funkcije, za potrebe računskih vežbi iz ovog kursa u svrhu minimizacije logičkih funkcija koristićemo metodu baziranu na Karnoovim kartama. Zbog toga, za potpuno razumevanje primera koji slede, neophodno je dobro poznavanje ove oblasti koja je odrađena na časovima predavanja.

Pored poznavanja funkcionalnosti osnovnih logičkih kola, kao i poznavanja metoda za minimizaciju logičkih funkcija, u toku izrade zadataka potrebno je poznavanje određenih algebarskih pravila koji važe u slučaju Bulove algebre. U tabeli 2 predstavljan su neka od osnovnih pravila Bulove algebre koja će biti korišćena kroz izradu zadataka.

Tabela 2: Neka od osnovnih pravila Bulove algebre

Broj pravila	I forma	II forma
1	$0 \cdot A = 0$	$0 + A = A$
2	$1 \cdot A = A$	$1 + A = 1$
3	$A \cdot A = A$	$A + A = A$
4	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
5	$A \cdot B = B \cdot A$	$A + B = B + A$
6	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$
7	$A + B \cdot C = (A + B) \cdot (A + C)$	$A \cdot (B + C) = A \cdot B + A \cdot C$
8	$A \cdot (A + B) = A$	$A + A \cdot B = A$
9	$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \cdot \bar{B}$
10	$A = \overline{\bar{A}}$	

U drugom delu računskih vežbi obrađivaćemo jednu vrstu hazarda karakterističnih za sekvencijalne mreže. Ta vrsta hazarda naziva statički hazardi i manifestuje se pojavom lažnih nula i jedinica na izlazu logičkog kola. U okiru praktičnih primera predstavimo pristup koji se može iskoristiti za uklanjanje statičkih hazarda. Međutim, treba imati u vidu da se u većini praktičnih situacija statički hazardi rešavaju korišćenjem metoda simulacije digitalnih kola.

2 Zadaci

2.1 Zadatak 1

Funkciju

$$Y = (A + C + D) \cdot (\bar{A} + C + \bar{D}) \cdot (B + \bar{C}) \quad (2.1)$$

realizovati u što minimalnijoj formi:

- Korišćenjem osnovnih logičkih kola sa proizvoljnim brojem ulaza
- Korišćenjem isključivo NI logičkih kola sa proizvoljnim brojem ulaza
- Korišćenjem isključivo NI logičkih kola sa dva ulaza
- Korišćenjem isključivo NILI logičkih kola sa dva ulaza

Rešenje:

a) Pošto se u okviru postavke zadatka traži realizacija u što minimalnijoj formi neophodno je izvršiti minimizaciju logičke funkcije korišćenjem nekih od dostupnih metoda za minimizaciju. U okviru ovog kursa, koristićemo metodu Karnoovih karti. Rezultat dobijen primenom ove metode garantuje minimalnost po pitanju broja logičkih kola ali i po pitanju broja ulaza.

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	0	1	0	0
01	0	1	1	1
11	1	0	1	1
10	1	0	0	0

Slika 1: Sadržaj Karnoove karte za funkciju (2.1)

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	0	1	0	0
01	0	1	1	1
11	1	0	1	1
10	1	0	0	0

(a) Proizvod zbirova

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	0	1	0	0
01	0	1	1	1
11	1	0	1	1
10	1	0	0	0

(b) Zbir proizvoda

Slika 2: Sadržaj Karnoove karte za funkciju (2.1), gde je a) označena oblast za minimizaciju u formi proizvoda zbirova b) označena oblast za minimizaciju u formi zbira proizvoda

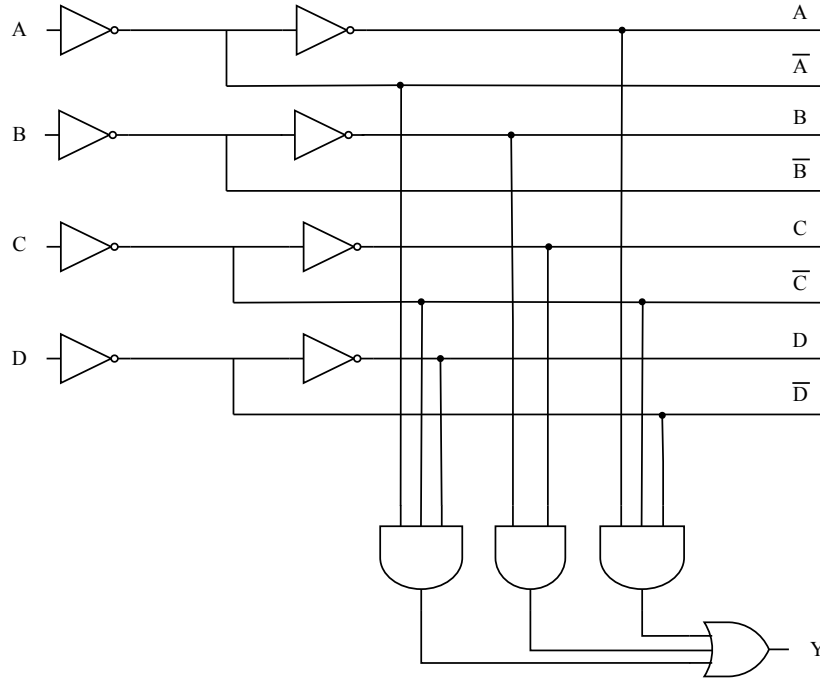
Nakon minimizacije dobijaju se minimalne funkcije u formi zbira proizvoda (2.2) i u formi proizvoda zbirova (2.3). Za realizaciju funkcije (2.2) potrebna su 3 I i 2 ILI logička kola. Isti broj

kola potreban je i za realizaciju funkcije (2.3). Dakle, obe minimalne forme daju isti broj logičkih kola.

$$Y_{MIN_ZP} = \bar{A}\bar{C}D + A\bar{C}\bar{D} + BC \quad (2.2)$$

$$Y_{MIN_PZ} = (A + C + D)(\bar{A} + C + \bar{D})(B + \bar{C}) \quad (2.3)$$

Na slici 3 je predstavljena realizacija funkcije (2.2).



Slika 3: Implementacija funkcije (2.1) u formi zbira proizvoda.

b) U okviru ove tačke zahtevana je implementacija funkcije (2.1) korišćenjem isključivo NI kola. Sistematski pristup za implementaciju funkcije koja sadrži isključivo NI kola podrazumeva dva koraka:

- 1) Transformaciju minimalne funkcije u formi zbira proizvoda u algebarski zapis koji sadrži isključivo komplementirane proizvode
- 2) Šemu koja sadrži isključivo NI kola a koja se dobija na osnovu algebarskog izraza dobijenog u koraku 1

Korak (1) obuhvata niz transformacija koje za cilj imaju dobijanje algebarskog zapisa u kome isključivo figurišu komplementirani proizvodi. U slučaju da se zahteva realizacija isključivo korišćenjem NI kola sa n ulaza, proizvodi moraju sadržati tačno n činioca. U okviru ove tačke broj ulaza nije definisan tako da ne postoji ograničenje po pitanju broja dostupnih ulaza u NI logičko kolo. Kao polazni izraz, nad kojim će se vršiti dalje transformacije, koristi se formazbira proizvoda funkcije dobijene primenom Karnoove karte. Za slučaj funkcije (2.1) polazni izraz predstavlja (2.2) odnosno zbir proizvoda. Na osnovu pravila Bulove algebre, koji kaže da se vrednost funkcije ne menja dvostrukim komplementiranjem iste, dobijamo:

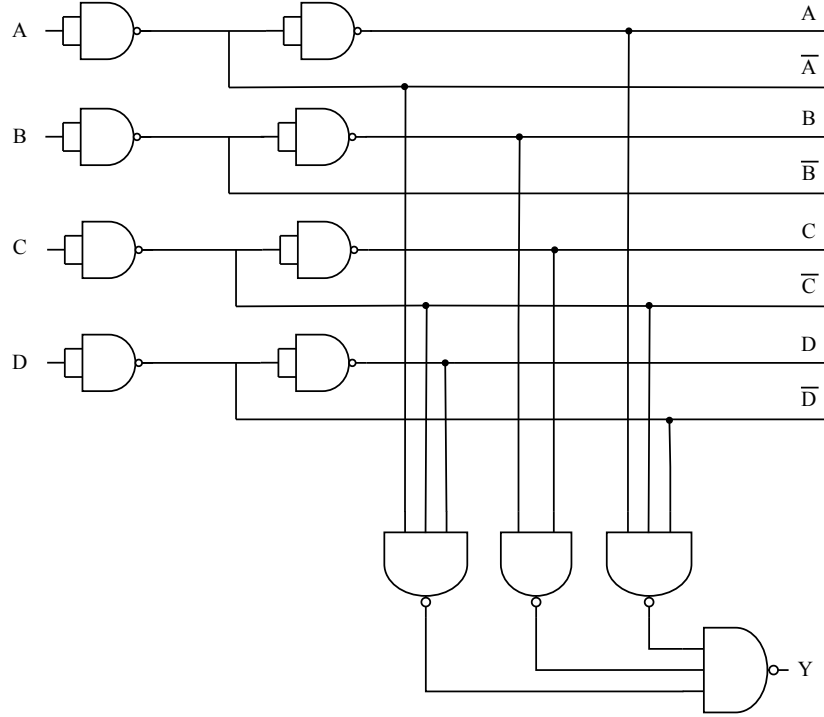
$$Y = \bar{\bar{Y}} = \bar{\bar{\bar{A}\bar{C}D + A\bar{C}\bar{D} + BC}} \quad (2.4)$$

Daljim transformacija kao krajnji rezultat dobijamo:

$$Y = \bar{\bar{Y}} = \bar{\bar{\bar{A}\bar{C}D} \cdot \bar{\bar{A}\bar{C}\bar{D}} \cdot \bar{\bar{B}C}} \quad (2.5)$$

Iz funkcije (2.5) možemo uočiti da u njoj figurišu isključivo komplementirani proizvodi, tačnije četiri takva proizvoda (crveni – $\overline{A}\overline{C}D$; plavi – $A\overline{C}\overline{D}$; zeleni – $\overline{B}C$; i na kraju globalni komplement koji predstavlja celu funkciju).

Na slici 4 je predstavljena implementacija funkcije (2.5).



Slika 4: Implementacija funkcije (2.1) korišćenjem isključivo NI kola sa proizvoljnim brojem ulaza.

c) U okviru ove tačke se takođe zahteva realizacija funkcije (2.1) korišćenjem **isključivo dvoulaznih NI** kola. Koraci (1) i (2) su identični kao u prethodnoj tački što podrazumeva da i ovde polazimo od algebarskog zapisa funkcije (2.1) u formi zbira proizvoda (2.2). Nakon niza transformacija (2.6)-(2.9) dobijamo krajnji izraz (2.10).

$$Y = \overline{C}(\overline{A}D + A\overline{D}) + BC \quad (2.6)$$

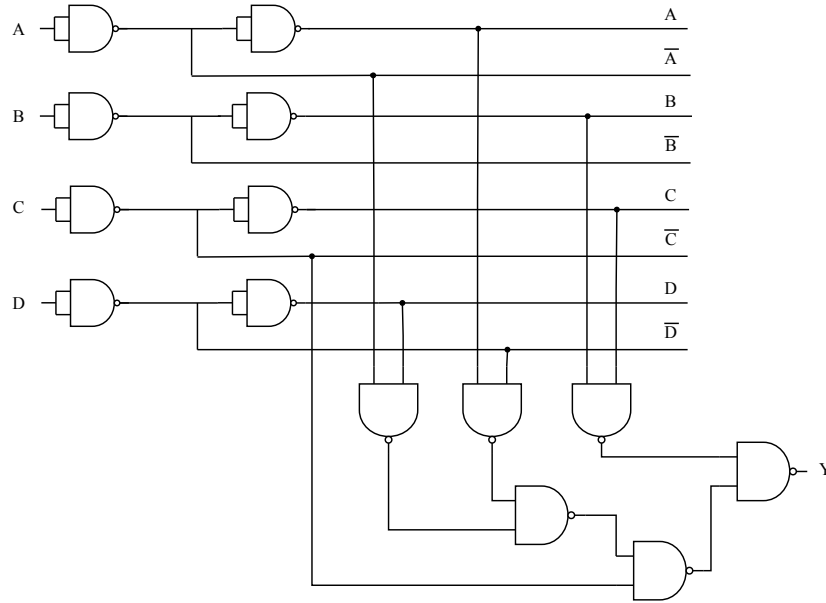
$$Y = \overline{\overline{\overline{C}(\overline{A}D + A\overline{D}) + BC}} \quad (2.7)$$

$$Y = \overline{\overline{\overline{\overline{C}(\overline{A}D + A\overline{D})} \cdot \overline{BC}}} \quad (2.8)$$

$$Y = \overline{\overline{\overline{\overline{\overline{C}(\overline{A}D + A\overline{D})} \cdot \overline{BC}}}} \quad (2.9)$$

$$Y = \overline{\overline{\overline{\overline{\overline{C}(\overline{A}D + A\overline{D})} \cdot \overline{BC}}}}} \quad (2.10)$$

Na slici 5 je predstavljena implementacija funkcije (2.10).



Slika 5: Implementacija funkcije (2.1) korišćenjem isključivo dvoulaznih NI kola.

d) U okviru ove tačke zahtevana je implementacija funkcije (2.1) korišćenjem isključivo dvoulaznih NILI kola. Sistematski pristup za implementaciju funkcije koja sadrži isključivo dvoulazna NILI kola podrazumeva sledeća dva koraka:

- 1) Transformaciju minimalne funkcije u formi proizvoda zbirova u algebarski zapis koji sadrži isključivo komplementirane zbirova
- 2) Šemu koja sadrži isključivo NILI kola a koja se dobija na osnovu algebarskog izraza dobijenog u koraku 1

Korak (1) obuhvata niz transformacija koje za cilj imaju dobijanje algebarskog zapisa u kome isključivo figurišu komplementirani zbirovi sa tačno dva sabirka. Kao polazni izraz, nad kojim će se vršiti dalje transformacije, koristi se forma proizvoda zbirova funkcije dobijene primenom Karnoove karte. Za slučaj funkcije (2.1) polazni izraz predstavlja (2.3) ili ponovljeno ispod:

$$Y = (A + C + D)(\bar{A} + C + \bar{D})(B + \bar{C}) \quad (2.11)$$

Na osnovu primena pravila 7 Bulove algebre definisanog u Tabeli 2 moguće je transformisati (2.11) u:

$$Y = (C + (A + D)(\bar{A} + \bar{D}))(B + \bar{C}) \quad (2.12)$$

Duplim komplementiranje izraza (2.12) dobija se

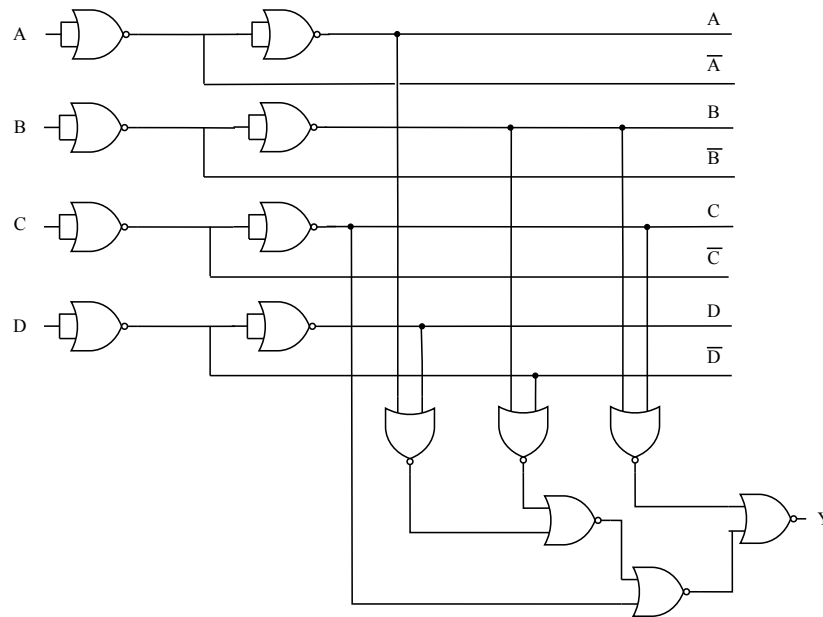
$$Y = \bar{\bar{Y}} = \overline{\overline{(C + (A + D)(\bar{A} + \bar{D}))(B + \bar{C})}} \quad (2.13)$$

$$Y = \overline{\overline{(C + (A + D)(\bar{A} + \bar{D}))} + \overline{(B + \bar{C})}} \quad (2.14)$$

$$Y = \overline{\overline{(C + (A + D)(\bar{A} + \bar{D}))} + \overline{(B + \bar{C})}} \quad (2.15)$$

$$Y = \overline{\overline{(C + (\bar{A} + \bar{D}) + (A + D))} + \overline{(B + \bar{C})}} \quad (2.16)$$

Na slici 6 je predstavljena implementacija funkcije (2.16).



Slika 6: Implementacija funkcije (2.1) korišćenjem isključivo dvoulaznih NILI kola.

VHDL kod koji implementira funkcije (2.2) i (2.3).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zadatak is
    port (
        A, B, C, D : in  STD_LOGIC;
        Y_ZP, Y_PZ : out STD_LOGIC
    );
end zadatak;

architecture Behavioral of zadatak is
    -- interni signali
    signal A_n, C_n, D_n : STD_LOGIC;
begin
    -----
    -- Invertovane vrednost signala
    -----

    A_n <= not A;
    C_n <= not C;
    D_n <= not D;

    -----
    --  $Y_{ZP} = A_n C_n D + A C_n D_n + BC$ 
    -----

    Y_ZP <= (A_n and C_n and D) or (A and C_n and D_n) or (B and C);

    -----
    --  $Y_{PZ} = (A + C + D)(A_n + C + D_n)(B + C_n)$ 
    -----

    Y_PZ <= (A or C or D) and (A_n or C or D_n) and (B or C_n);
end Behavioral;
```

Testbench koji simulira date funkcije je dat [OVDE](#).

2.2 Zadatak 2

- a) Odrediti funkcionalnu tabelu za funkciju $Z = f(A, B, C, D)$, gde je Z jednako 1 kada u zapisu četvorobitnog broja $ABCD$ postoje barem dve susedne nule, dok je u suprotnom Z jednako 0 (na primer, kada je $ABCD = 0000$ tada je $Z = 1$, dok kada je $ABCD = 0101$ tada je $Z = 0$).
- b) Projektovati kombinacionu mrežu koja realizuje funkciju Z ukoliko su dostupna logička kola proizvoljnog tipa.

Napomena. Težiti da broj upotrebljenih logičkih kola bude minimalan.

Rešenje:

- a) Sadržaj funkcionalne tabele za funkciju Z je:

Tabela 3: Funkcionalna tabela za $Z = f(A, B, C, D)$

A	B	C	D	Z
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

b) Pošto se postavkom zadatka zahteva da broj upotrebljenih kola bude minimalan, realizaciju ove logičke funkcije potrebno je započeti korišćenjem metode minimizacije bazirane na upotrebi Karnoovih karti. Sadržaj Karnove karte za funkcionalnu tabelu 3 iz prethodne tačke je prikazan na slici 7.

Funkciju Z ćemo najpre izraziti u obe forme (zbir proizvoda i proizvod zbirova) a zatim ćemo analizom utvrditi koja forma zahteva manji broj logičkih kola.

Na osnovu slike 8 dobijamo sledeće funkcije:

$$Y_{MIN_ZP} = \bar{C}\bar{D} + \bar{A}\bar{B} + \bar{B}\bar{C} \quad (2.17)$$

$$Y_{MIN_PZ} = (\bar{B} + \bar{C})(\bar{B} + \bar{D})(\bar{A} + \bar{C}) \quad (2.18)$$

Na osnovu dobijenih logičkih funkcija možemo zaključiti da je isti broj logičkih kola potreban za realizaciju obe forme logičke funkcije Z .

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	1	1	1	1
01	1	0	0	0
11	1	0	0	0
10	1	1	0	0

Slika 7: Sadržaj Karnoove karte za funkciju Z .

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	1	1	1	1
01	1	0	0	0
11	1	0	0	0
10	1	1	0	0

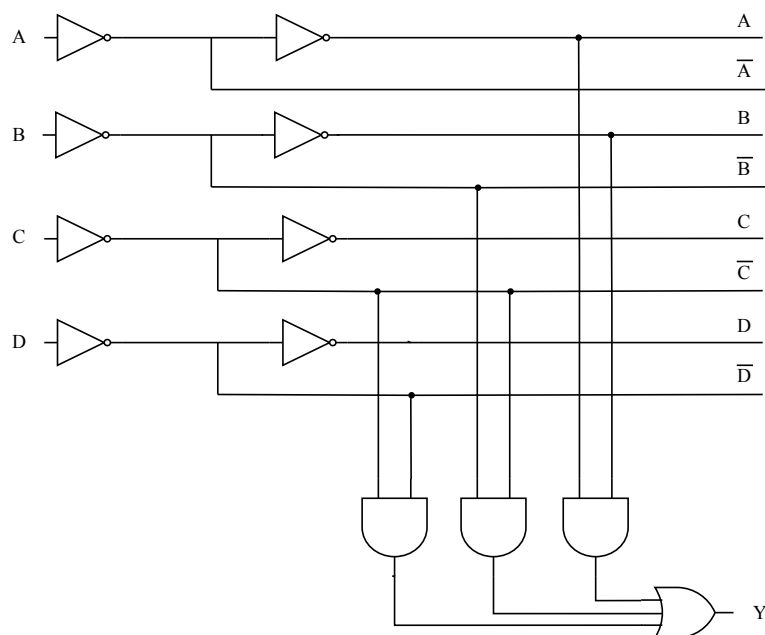
(a) Proizvod zbirova

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	1	1	1	1
01	1	0	0	0
11	1	0	0	0
10	1	1	0	0

(b) Zbir proizvoda

Slika 8: Karnoova mape za funkciju Z gde je a) označena oblast za minimizaciju u formi proizvoda zbirova b) označena oblast za minimizaciju u formi zbira proizvoda.

Na slici 9 je prikazana implementacija funkcije (2.17).

**Slika 9:** Implementacija funkcije Z korišćenjem forme zbira proizvoda.

VHDL kod koji implementira funkcije (2.17) i (2.18).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zadatak is
    port (
        A, B, C, D : in  STD_LOGIC;
        Y_ZP, Y_PZ : out STD_LOGIC
    );
end zadatak;

architecture Behavioral of zadatak is
    -- interni signali
    signal A_n, B_n, C_n, D_n : STD_LOGIC;
begin
    -----
    -- Invertovane vrednost signala
    -----
    A_n <= not A;
    B_n <= not B;
    C_n <= not C;
    D_n <= not D;
    -----
    --  $Y_{ZP} = C_n D_n + A_n B_n + B_n C_n$ 
    -----
    Y_ZP <= (C_n and D_n) or (A_n and B_n) or (B_n and C_n);
    -----
    --  $Y_{PZ} = (B_n + C_n)(B_n + D_n)(A_n + C_n)$ 
    -----
    Y_PZ <= (B_n or C_n) and (B_n or D_n) and (A_n or C_n);
end Behavioral;
```

Testbench koji simulira funkcije (2.17) i (2.18) je dat [OVDE](#).

2.3 Zadatak 3

Projektovati kombinacionu mrežu koja realizuje izlaz $C(C_3C_2C_1C_0) = A(A_1A_0) \cdot B(B_1B_0)$, gde su A i B neoznačeni binarni brojevi. Kombinacionu mrežu je potrebno realizovati korišćenjem minimalnog broja osnovnih logičkih kola sa proizvoljnim brojem ulaza.

Rešenje:

Prvi korak u rešavanju ovog zadatka predstavlja kreiranje sadržaja funkcionalne tabele:

Tabela 4: Funkcionalna tabela za $C = A \cdot B$

A_1	A_0	B_1	B_0	C_3	C_2	C_1	C_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	1	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1

Nakon kreiranja funkcionalne tabele, primenom metode optimizacije bazirane na Karnoovim kartama, potrebno je realizovati minimalnu funkciju za svaki od izlaznih signala. Sadržaj Karnoovih karti je prikazan na slikama 10 i 11.

B_1B_0 A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	1	0
10	0	0	0	0

(a) Karnoova mapa za C_3

B_1B_0 A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	1
10	0	0	1	1

(b) Karnoova mapa za C_2

Slika 10: Karnoove karte za C_3 i C_2 proizvoda $C = A \cdot B$, sa grupisanim jedinicama (ZP).

$\begin{matrix} B_1B_0 \\ A_1A_0 \end{matrix}$	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	1	0	1
10	0	1	1	0

(a) Karnoova mapa za C_1

$\begin{matrix} B_1B_0 \\ A_1A_0 \end{matrix}$	00	01	11	10
00	0	0	0	0
01	0	1	1	0
11	0	1	1	0
10	0	0	0	0

(b) Karnoova mapa za C_0 **Slika 11:** Karnoove karte za C_1 i C_0 proizvoda $C = A \cdot B$, sa grupisanim jedinicama (ZP).

Na osnovu sadržaja Karnoovih karti, prikazanih na slikama 10 i 11, moguće je realizovati svaku od izlaznih funkcija:

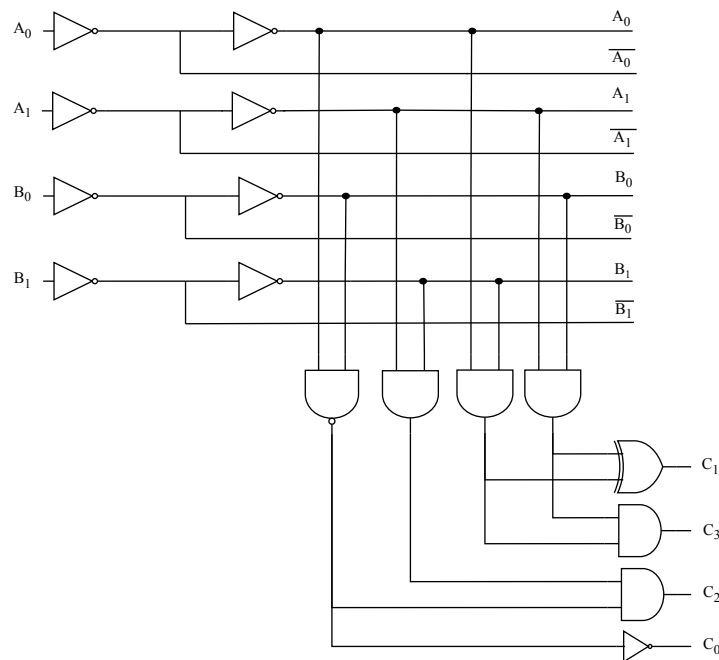
$$C_3 = A_1 A_0 B_1 B_0 \quad (2.19)$$

$$C_2 = A_1 B_1 \bar{B}_0 + \bar{A}_0 A_1 B_1 = A_1 B_1 (\bar{B}_0 + \bar{A}_0) = A_1 B_1 \overline{A_0 B_0} \quad (2.20)$$

$$\begin{aligned} C_1 &= A_1 \bar{B}_1 B_0 + A_1 \bar{A}_0 B_0 + \bar{A}_1 A_0 B_1 + A_0 B_1 \bar{B}_0 \\ &= A_1 B_0 (\bar{A}_0 + \bar{B}_1) + A_0 B_1 (\bar{A}_1 + \bar{B}_0) \\ &= A_1 B_0 (\overline{A_0 B_1}) + A_0 B_1 (\overline{A_1 B_0}) = A_1 B_0 \oplus A_0 B_1 \end{aligned} \quad (2.21)$$

$$C_0 = A_0 B_0 \quad (2.22)$$

Na slici 11 je predstavljena implementacija funkcije $C = A \cdot B$.

**Slika 12:** Implementacija funkcije $C = A \cdot B$.

VHDL kod koji implementira funkciju $C = A \cdot B$.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zadatak is
    port (
        A, B: in  STD_LOGIC_VECTOR(1 downto 0);
        C   : out STD_LOGIC_VECTOR(3 downto 0)
    );
end zadatak;

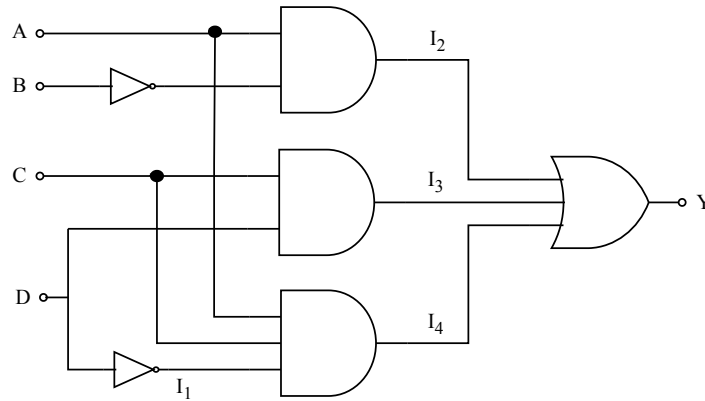
architecture Behavioral of zadatak is
begin
    -----
    -- CO = A0 B0
    -----
    C(0) <= A(0) and B(0);
    -----
    -- C1 = (A1 B0) xor (A0 B1)
    -----
    C(1) <= (A(1) and B(0)) xor (A(0) and B(1));
    -----
    -- C2 = (A1 B1) (A0 nand B0)
    -----
    C(2) <= (A(1) and B(1)) and (A(0) nand B(0));
    -----
    -- C3 = A1 A0 B1 B0
    -----
    C(3) <= A(1) and A(0) and B(1) and B(0);
end Behavioral;
```

Testbench koji simulira funkciju $C = A \cdot B$ je dat [OVDE](#).

2.4 Zadatak 4

Za kolo predstavljeno na slici 13:

- Odrediti logičku funkciju
- Nacrtati vremenski dijagram prelaza ABCD: 1111 → 1110 za signale D , I_1 , I_2 , I_3 , I_4 i Y ako je kašnjenje svakog logičkog kola jednako T . Komentarisati dobijene vremenske dijagrame.
- Predložiti realizaciju koja rešava probleme uočene pod b)



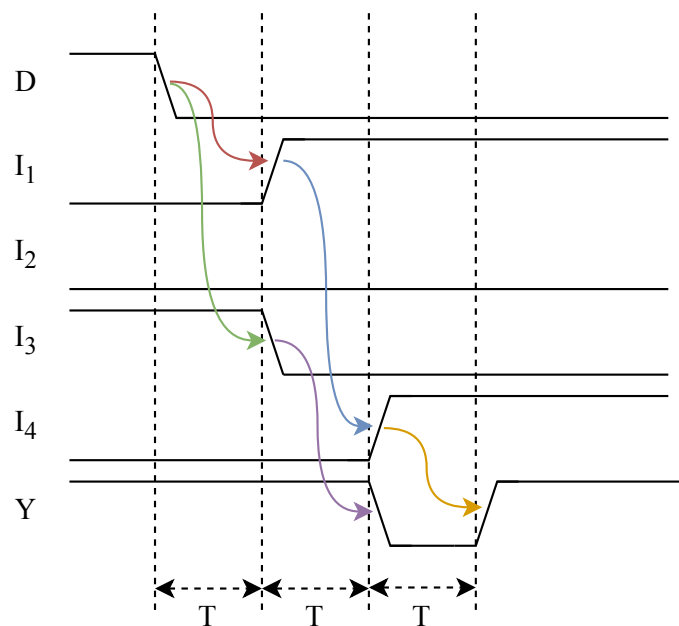
Slika 13: Šema logičkog kola uz zadatak 2.4.

Rešenje:

- Logička funkcija kojom se opisuje kolo prikazano na slici 13 je:

$$Y = A\bar{B} + CD + AC\bar{D} \quad (2.23)$$

- Vremenski dijagram prelaza ABCD: 1111 → 1110 je prikazan na slici 13



Slika 14: Vremenski dijagram prelaza ABCD: 1111 → 1110 za logičko kolo sa slike 13.

Ako posmatramo vrednost funkcije definiše jednačina (2.23) videćemo da za obe kombinacije ulaza $ABCD = 1111$ i $ABCD = 1110$ dobijamo istu vrednost. Međutim, uvidom u vremenski dijagram možemo uočiti da postoji jedan vremenski interval, jednak kašnjenju logičkog kola, u okviru koga vrednost izlaza Y postaje jednak logičkoj nuli. Takvo ponašanje nije poželjno u logičkim kolima koje realizujemo jer se potencijalno može degradirati funkcionalnost ostatka digitalnog sistema koji koristi takvu realizaciju. Zbog toga se postavlja pitanje kako rešiti uočeni problem u ovom konkretnom slučaju i još mnogo bitnije pitanje, kako uopšteno detektovati da postoji problem u nekom kolu a zatim ga sistematično rešavati?

Na putu do odgovora na postavljena pitanja, potrebno je analizirati sadržaj Karnoove karte kao i odgovarajućih oblasti koje u toj Karnoovoj karti pokriva funkcija (2.23). Na slici 14 je predstavljen sadržaj Karnoove karte sa naznačenim odgovarajućim oblastima:

$\begin{array}{c} CD \\ AB \end{array}$	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	1
10	1	1	1	1

Slika 15: Sadržaj Karnoove karte za funkciju (2.23).

Na osnovu sadržaja Karnoove karte ali i prikazanih oblasti možemo uočiti da se posmatrani prelaz, koji dovodi do pojave logičke nule, nalazi između dve nepreklopljene oblasti u Karnoovoj karti. U opštem slučaju se ovaj problem naziva problem lažne nule (ili lažne jedinice za slučaj funkcije izražene koristeći formu PZ) a javlja se, kao i ovom konkretnom slučaju, pri prelazima iz oblasti u Karnoovoj karti koje nisu preklapljene. U opštem slučaju problem pojave lažne nule (ili lažne jedinice za slučaj funkcije izražene koristeći formu PZ) se može javiti u oba smera.

Za samostalni rad: Nacrtati vremenske dijagrame za prelaz $ABCD: 1110 \rightarrow 1111$.

.....
c) Jedan od načina za rešavanje ovog problema predstavlja preklapanje prelaza koje potencijalno može dovesti do pojave lažne nule ili lažne jedinice. Za slučaj u ovom zadatku dobijamo oblasti kao što je prikazano na slici 16.

Na osnovu Karnoove karte sa slike 16, izvodimo sledeću funkciju:

$$Y = A\bar{B} + CD + AC \quad (2.24)$$

Za samostalni rad: Nacrtati vremenske dijagrame za funkciju (2.24) za prelaz $ABCD: 1110 \rightarrow 1111$.

Poređenjem funkcija (2.24) i (2.23) možemo zaključiti da je primena metode rešavanja statičkog hazarda (pojava lažne nule ili lažne jedinice) u ovom konkretnom slučaju doprinela i dodatnoj minimizaciji logičke funkcije (2.23) što **NE MORA** uvek biti slučaj.

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	1
10	1	1	1	1

Slika 16: Karnoove karta za funkciju (2.23).

VHDL kod koji implementira funkciju (2.23) sa kašnjenjem T.

Napomena: Kašnjenja dodata u kod se koriste samo za simulacije!

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zadatak is
  generic (
    T : time := 1 ns -- globalni delay parametar
  );
  port (
    A, B, C, D : in STD_LOGIC;
    Y : out STD_LOGIC
  );
end zadatak;

architecture Behavioral of zadatak is
  -- interni signali
  signal B_inv, I1 : STD_LOGIC;
  signal I2, I3, I4 : STD_LOGIC;
begin
  -----
  -- B_inv = inv B, I1 = inv D
  B_inv <= (not B) after T;
  I1 <= (not D) after T;
  -----
  -- I2 = A B_inv
  I2 <= (A and B_inv) after T;
  -----
  -- I3 = C D
  I3 <= (C and D) after T;
  -----
  -- I4 = A and C and I1 (A C \bar{D})
  I4 <= (A and C and I1) after T;
  -----
  -- Y = I2 + I3 + I4
  Y <= (I2 or I3 or I4) after T;
end Behavioral;

```

Testbench koji simulira prelazak $ABCD = 1111 \rightarrow ABCD = 1110$ je dat OVDE.

2.5 Zadatak 5

Data je funkcija

$$Y = \bar{A}\bar{C} + CD + BCD\bar{D} \quad (2.25)$$

- Odrediti pri kojim prelazima postoji mogućnost generisanja lažnih nula na izlazu kola.
- Predstaviti funkciju (2.25) u minimalnoj formi
- Ukoliko u realizaciji b) postoji mogućnost generisanja lažnih nula, modifikovati funkciju tako da se to spreči

Rešenje:

Na slici 17 je prikazan sadržaj Karnoove karte za funkciju (2.25).

CD \ AB	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	0	1	1
10	0	0	1	0

Slika 17: Sadržaj Karnoove karte za funkciju (2.25).

Na osnovu obeleženih kontura koje odgovaraju pojedinačnim logičkim proizvodima iz logičke funkcije (2.25), vidimo da funkcija nije data u minimalnoj formi. Takođe, uočavamo potencijalne prelaze koji mogu dovesti do generisanja lažne nule na izlazu kola. To su svi prelazi između dve susedne konture, koji se mogu desiti pri promeni najviše jednog od ulaznih signala. Na slici 17 su ovi prelazi označeni parom strelica na levu i desnu stranu. Prema tabeli sa slike 17 uočavamo da postoji ukupno 10 prelaza koji mogu dovesti do pojave lažne nule na izlazu kola. Kao što je obeleženo u Karnoovoj karti na slici 17, polja $ABCD = 0100$ i $ABCD = 0110$ predstavljaju susedna polja u Karnoovoj karti.

b) Na slici 18 je predstavljen sadržaj Karnoove karte sa označenim oblastima za slučaj minimalne forme funkcije (2.25). Kao što se sa slike 18 može primetiti, minimalna forma funkcije i dalje sadrži prelaze koji potencijalno mogu uzrokovati pojavu lažnih nula i jedinica.

Na osnovu slike 18 dobijamo funkciju (2.25) u minimalnoj formi:

$$Y = \bar{A}\bar{C} + CD + BC \quad (2.26)$$

c) Na slici 19 je prikazana Karnoova karta sa naznačenim oblastima koje omogućavaju realizaciju funkcije u formi koja neće dovesti do generisanja lažnih nula.

Na osnovu Karnoove karte, prikazane na slici 19, moguće je izvesti sledeći zapis funkcije:

$$Y = \bar{A}\bar{C} + CD + BC + \bar{A}B + \bar{A}D \quad (2.27)$$

Dakle, polazeći od funkcije (2.25) najpre smo u tački b) izveli izraz koji predstavlja funkciju (2.25) u minimalnoj formi (2.26). Međutim tako dobijena forma je i dalje potencijalno generisala lažne nule. Da bi smo sprečili generisanje lažnih nula, na osnovu preklapanja oblasti u

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	0	1	1
10	0	0	1	0

Slika 18: Sadržaj Karnoove karte za slučaj minimalne forme funkcije (2.25) sa označenim prelazima koji mogu uzrokovati pojavu lažnih nula.

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	0	1	1
10	0	0	1	0

Slika 19: Karnoova karta sa naznačenim oblastima koje omogućavaju realizaciju funkcije (2.25) u formi koja neće dovesti do generisanja lažnih nula.

Karnoovoj karti dobili smo formu (2.27). Za razliku od (2.26), forma predstavljena u okviru (2.27) sadrži veći broj logičkih kola ali sprečava pojavu lažnih nula.

VHDL kod koji implementira funkcije (2.26) i (2.27) sa kašnjenjem T.

Napomena: Kašnjenja dodata u kod se koriste samo za simulacije!

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zadatak is
    generic (
        T : time := 1 ns    -- globalni delay parametar
    );
    port (
        A, B, C, D: in  STD_LOGIC;
        Y_min, Y_no_hazard : out STD_LOGIC
    );
end zadatak;

architecture Behavioral of zadatak is
    -- interni signali
    signal A_inv, C_inv : STD_LOGIC;
    signal I1, I2, I3, I4, I5 : STD_LOGIC;
begin
    -----
    -- Invertovane vrednosti
    A_inv <= (not A) after T;
    C_inv <= (not C) after T;
    -----
    -- I1 = A_n C_n
    I1 <= (A_inv and C_inv) after T;
    -----
    -- I2 = C D
    I2 <= (C and D) after T;
    -----
    -- I3 = B C
    I3 <= (B and C) after T;
    -----
    -- I4 = A_n B
    I4 <= (A_inv and B) after T;
    -----
    -- I5 = A_n D
    I5 <= (A_inv and D) after T;
    -----
    -- Y_min = A_n C_n + C D + BC = I1 + I2 + I3
    Y_min <= (I1 or I2 or I3) after T;
    -----
    -- Y_no_hazard = A_n C_n + CD + BC + A_nB + A_nD
    --               = I1 + I2 + I3 + I4 + I5
    Y_no_hazard <= (I1 or I2 or I3 or I4 or I5) after T;
end Behavioral;

```

Testbench koji simulira funkcije (2.26) i (2.27) je dat [OVDE](#).

2.6 Zadatak 6

Data je funkcija u formi zbira logičkih proizvoda:

$$Y = \bar{B}\bar{C}\bar{D} + \bar{A}\bar{C}D + \bar{A}C\bar{D} \quad (2.28)$$

- Logičku funkciju (2.28) predstaviti u formi proizvoda logičkih zbirova. Da li, i pri kojim prelazima, postoji mogućnost generisanja lažnih jedinica na izlazu kola?
- Modifikovati logičku funkciju tako da ne postoji mogućnost generisanja lažnih jedinica

Rešenje:

- Na slici 20 je predstavljen sadržaj Karnoove karte dobijen na osnovu funkcije (2.28).

$\begin{smallmatrix} C/D \\ A/B \end{smallmatrix}$	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	0	1	0	0
10	1	1	0	0

Slika 20: Sadržaj Karnoove karte za funkciju (2.28).

Na osnovu Karnoove karte prikazane na slici 20, određujemo logičku funkciju izlaza kola u minimalnoj formi u obliku proizvoda logičkih zbirova:

$$Y = (\bar{B} + C + D)(A + \bar{D})(\bar{A} + \bar{C}) \quad (2.29)$$

Sa slike 20 uočavamo da logička funkcija (2.29) u traženom obliku ima više potencijalno opasnih prelaza u pogledu pojave statičkih hazarda, odnosno generisanja lažne jedinice na izlazu kola, u odnosu na realizaciju mreže u obliku zbira logičkih proizvoda. Na slici 21 su obeleženi svi prelazi koji mogu dovesti do pojave lažnih jedinica

$\begin{smallmatrix} C/D \\ A/B \end{smallmatrix}$	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	0	1	0	0
10	1	1	0	0

Slika 21: Karnoova karta funkcije (2.28) sa naznačenim prelazima koji dovode do pojave lažne jedinice.

.....

$\begin{array}{c} CD \\ AB \end{array}$	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	0	1	0	0
10	1	1	0	0

Slika 22: Karnoova karta funkcije (2.28) sa naznačenim prelazima koji ne dovode do pojave lažnih jedinica.

b) Nova forma funkcije (2.28) koja ne dovodi do pojave lažnih jedinica dobija se na osnovu Karnoove karte prikazane na slici 22

Na osnovu Karnoove karte prikazane na slici 22 izvodimo sledeću funkciju:

$$Y = (\bar{B} + C + D)(A + \bar{D})(\bar{A} + \bar{C})(\bar{C} + \bar{D})(A + \bar{B} + C)(\bar{A} + \bar{B} + D) \quad (2.30)$$

VHDL kod koji implementira funkcije (2.29) i (2.30) sa kašnjenjem T.

Napomena: Kašnjenja dodata u kod se koriste samo za simulacije!

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zadatak is
  generic (
    T : time := 1 ns -- globalni delay parametar
  );
  port (
    A, B, C, D: in STD_LOGIC;
    Y_min, Y_no_hazard : out STD_LOGIC
  );
end zadatak;

architecture Behavioral of zadatak is
  -- interni signali
  signal A_inv, B_inv, C_inv, D_inv : STD_LOGIC;
  signal I1, I2, I3, I4, I5, I6 : STD_LOGIC;
begin
  -----
  -- Invertovane vrednosti
  A_inv <= (not A) after T;
  B_inv <= (not B) after T;
  C_inv <= (not C) after T;
  D_inv <= (not D) after T;
  -----
  -- I1 = (B_n + C + D)
  I1 <= (B_inv or C or D) after T;
  -----
  -- I2 = (A + D_n)
  I2 <= (A or D_inv) after T;
  -----
  -- I3 = (A_n + C_n)
  I3 <= (A_inv or C_inv) after T;
  -----
  -- I4 = (C_n + D_n)
  I4 <= (C_inv or D_inv) after T;
  -----
  -- I5 = (A + B_n + C)
  I5 <= (A or B_inv or C) after T;
  -----
  -- I6 = (A_n + B_n + D)
  I6 <= (A_inv or B_inv or D) after T;
  -----
  -- Y_min = (B_n + C + D)(A + D_n)(A_n + C_n) = I1 I2 I3
  Y_min <= (I1 and I2 and I3) after T;
  -----
  -- Y_no_hazard =
  -- (B_n + C + D)(A + D_n)(A_n + C_n)(C_n + D_n)(A + B_n + C)(A_n + B_n + D)
  -- I1 I2 I3 I4 I4 I6
  Y_no_hazard <= (I1 and I2 and I3 and I4 and I5 and I6) after T;
end Behavioral;

```

Testbench koji simulira funkcije (2.29) i (2.30) je dat [OVDE](#).

3 Zadaci za samostalni rad

3.1 Zadatak 1

Projektovati kombinacionu mrežu koja na ulazima prihvata dva dvobitna neoznačena binarna broja, a na izlazima generiše binarni broj koji predstavlja zbir kvadrata ulaznih brojeva. Na raspolaganju su proizvoljna logička kola niskog stepena integracije. Težiti da broj upotrebljenih logičkih kola bude minimalan.

3.2 Zadatak 2

Projektovati kombinacionu mrežu koja za četvorobitni binarni broj na ulazu generiše binarni broj koji predstavlja broj pojavljivanja jedinice u ulaznom binarnom broju. Na raspolaganju su proizvoljna logička kola. Težiti da broj upotrebljenih logičkih kola bude minimalan.